

MacroFit Final Project Report

Nagur Pavan | PE6201 Emerging AI Technologies | 1 October 2026

Purpose This report explains the final product, the reasoning behind its hybrid architecture, the evidence obtained, and the limitations that remain before real-world use.

Executive Summary

MacroFit is a meal-planning prototype for adults who depend on prepared food from restaurants, food courts, or takeaway services. A user supplies a profile, daily budget, and ordinary food language. The system estimates general daily nutrition, produces a valid breakfast, lunch, and dinner shortlist, and selects one plan with an explanation. The project does not claim medical accuracy or current market prices. Its purpose is to test whether a transparent combination of deterministic Python and a rented language model can produce practical choices without giving the model control over arithmetic or safety rules.

The latest evaluation passed 19 of 20 pipeline cases at a provider-reported cost of US\$0.00546315. This shows that the architecture usually follows the intended route, including unusual inputs. It does not yet prove that the model improves human preference: five blind judgements were ties, and only cases where AI and the optimizer choose different plans can measure added value.

Problem and Intended Difference

People who buy most meals face a repeated decision across the day. They may know their budget and mood but do not want to calculate calories, compare nutrition, remember earlier recommendations, and inspect many menus. Existing calorie tools mainly record food after eating, while ordering platforms rank individual items without balancing the whole day. MacroFit combines these tasks into one bounded planning flow.

The practical difference is not automatic health advice. It is a clearer decision process. The user can express preferences such as spicy, vegetarian for lunch, light for dinner, portable, or not rice. MacroFit converts those words into constraints and preferences, proposes scheduled meals, remembers recommendations during the active session, and recalculates the rest of the day after confirmed intake. If the request is contradictory, unsafe, or impossible within budget, the system explains why it cannot proceed instead of presenting a misleading plan.

Technical and Business Reasoning

I separated work according to what each method can do reliably. Python uses the Mifflin St Jeor equation, activity multiplier, goal adjustment, and a documented macro split to estimate general targets. It joins menu and nutrition records by item ID, enumerates meal combinations, applies meal-slot and dietary filters, checks the budget, and requires at least 90 percent of the calorie, protein, and carbohydrate estimates. It also prevents same-day duplicates, excludes recommendations already shown during the session, and constructs a diverse shortlist. These operations are auditable and should not be delegated to a language model.

AI Layer 1 handles language that would be brittle to cover with a growing list of rules. It converts phrases about cuisine, comfort, spice, heaviness, portability, and food format into a closed JSON schema. Python validates the result before it affects planning. AI Layer 2 receives only valid shortlisted plans and chooses one based on subjective fit. It may return only an exact plan ID. Python checks that ID again. A third bounded call interprets actual-intake messages, while Python performs the nutrition and budget recalculation.

I rent the model through OpenRouter because training or hosting a foundation model would add cost and operational work without improving the course objective. The model is replaceable through configuration. This reduces lock-in, but provider rate limits, malformed output, and behaviour changes

remain external risks. Strict schemas, two-attempt retry, explicit technical errors, and output validation limit their effect.

Data and Responsible Use

The prototype contains 60 synthetic menu items from 10 synthetic outlets and 60 matching nutrition rows. Each menu item has price, cuisine, dietary type, spice, heaviness, format, portability, and preference tags. Nutrition rows contain calories, protein, carbohydrates, fat, fibre, sodium, source family, matching method, mapper, date, and confidence. The current distribution is 20 chicken, 19 vegetarian, 11 fish, 5 egg, and 5 beef items. Vegetarian is defined as no meat, fish, or egg.

This data validates joins and system behaviour; it does not represent current prices or medically verified nutrition. Confidence is high for 8 mappings, medium for 36, and low for 16. A wrong label can silently pass through correct code. Egg Fried Rice was initially labelled vegetarian, which exposed this risk. Yong Tau Foo item H04 still needs review because its generic nutrition match refers to fishcake. Real deployment would require measured portions, timestamped prices, verified dietary and allergen labels, and independent nutrition review.

Metrics and Final Outcomes

The primary pipeline metric is whether each test reaches its expected outcome. The 20 manually written cases include 12 normal requests and 8 outliers covering contradictions, unclear references, an infeasible budget, prompt attacks, a complex valid preference, and an extreme intake conflict. The latest run passed 19 cases, or 95 percent. It made 13 Layer 1 calls, 12 Layer 2 calls, and 25 API calls. The run used 25,997 prompt tokens and 2,606 completion tokens.

The provider-reported evaluation cost was US\$0.00546315. The calculated US\$0.00109263 figure is best described as cost per valid judged recommendation, not cost per AI win. AI and the optimizer selected different plans in 7 of the 12 Layer 2 cases and selected the same plan in 5. Five blind judgements were entered and all were ties. Because there were no AI-preferred wins, cost per AI-preferred win is currently unavailable.

Metric	Result	Interpretation
Pipeline pass rate	19 of 20	95 percent
Layer 1 calls	13	Language interpretation
Layer 2 calls	12	Subjective shortlist ranking
Different AI selections	7 of 12	Eligible for blind comparison
Evaluation cost	US\$0.00546315	Provider reported
Blind evidence	5 ties	No AI preference win yet

Evaluation Critique

The 95 percent pass rate is encouraging because the outliers test more than the happy path. Prompt-injection attempts were blocked before a model call. Contradictions and unclear references produced clarification, and the S\$2 case stopped as infeasible. The single failure was E06, which asked for spicy non-veg lunch and light vegetarian dinner. The deterministic precheck matched the word veg inside non-veg and incorrectly reported a lunch contradiction. This is a parser defect with a precise correction: standalone veg must not match when preceded by non-. The complete evaluation must be rerun after that correction.

The blind study is less mature. Constraint accuracy belongs to Python, so it cannot be used to claim that the model earns its place. Identical AI and optimizer selections should be recorded as same selection rather than blind ties. Only the seven changed selections can compare preference. Five judgements are also too few for a reliable conclusion. The honest finding is that the model produced valid subjective choices cheaply, but the available human evidence does not yet show that those choices are better.

Difficulties and Performance Tuning

The hardest design problem was preventing the model from doing work that Python already solves. Early versions risked presenting target compliance as an AI achievement. The final flow makes the optimizer the non-AI baseline and confines the model to language interpretation and preference ranking. Another difficulty was semantic ambiguity. Terms such as veg, non-veg, morning, that meal, and the other meal require either explicit mapping or clarification. Live actual-intake mode now asks up to two follow-up questions, while evaluation mode records clarification and moves on without interactive input.

Repeated recommendations revealed a second design weakness. A large menu does not create variety when the same few combinations always receive the best numeric score. I removed the unused ranking factor, retained a larger candidate pool, diversified the final shortlist, added a penalty for recently eaten food, and hard-excluded items already recommended in the active session. Model efficiency was improved by sending at most five compact plans, limiting output tokens, skipping Layer 2 when no subjective judgement is needed, and retrying malformed output only once.

Rough Edges and Future Path

The main rough edges are synthetic data, temporary Colab memory, incomplete blind evaluation, and a few rule definitions that still require review. Recommendation history disappears when the runtime restarts. The prototype does not certify allergens, connect to live menus, or support clinical diets. The language model may still misunderstand unusual phrasing even though schemas contain its output.

The next step is to correct E06, review H04, rerun all 20 cases, and complete blind judgements for every changed selection. A stronger pilot would persist user-controlled history, add an explicit egg preference, verify a smaller real menu, and log model version and dataset version with each result. The architecture should remain hybrid: Python should continue to own calculations and hard constraints, while AI should be retained only if a larger blind study shows that people prefer its choices often enough to justify its cost and complexity.